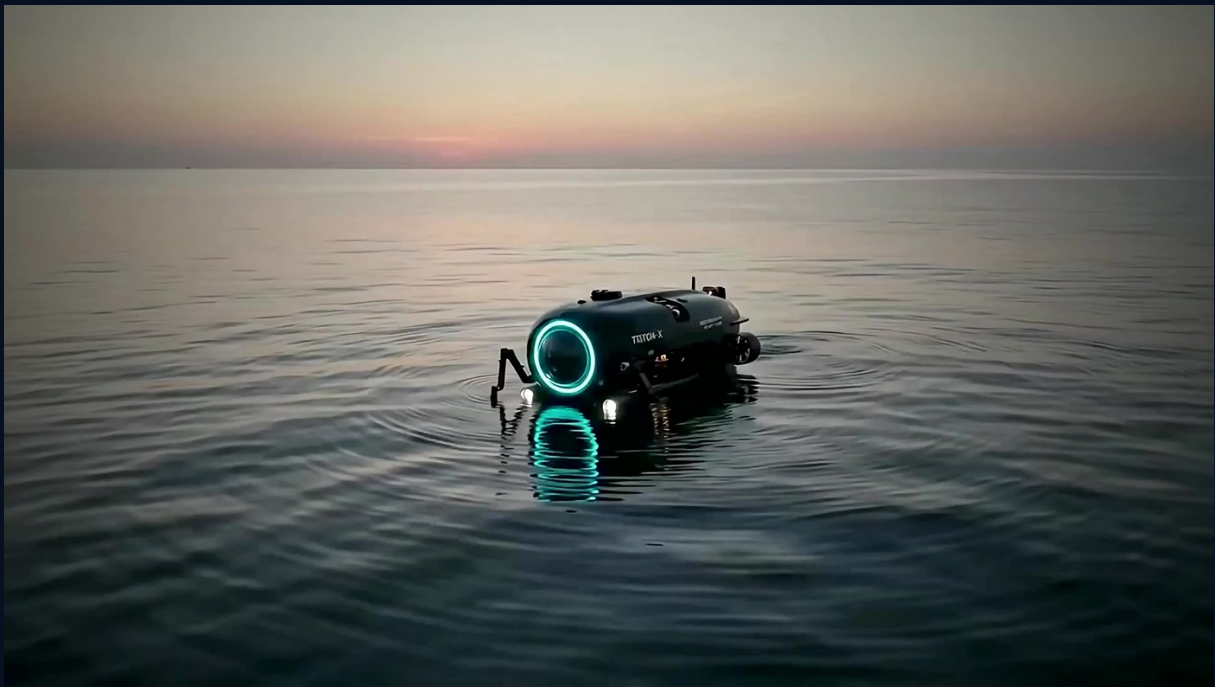


ABYSSAL VENTURES

A cinematic scroll-dive website

Technical Project Report



SCROLLING DOWN IS DESCENDING · 0 → 3,800 METRES

318-FRAME CANVAS SCRUB · VANILLA HTML / CSS / JS · NO BUILD STEP

PREPARED FOR GOPI PATEL • AUGUST 2026

01 — OVERVIEW

What this project is

ABYSSAL VENTURES is a single-page, cinematic marketing site for a fictional deep-sea expedition company that takes eight civilians a year to the ocean floor aboard its submersible, the **Triton-X**. The central idea is that **scrolling down is descending**: the page has no conventional scrolling content over a static background. Instead, a fixed full-viewport canvas plays a 3D submarine descent frame by frame, driven directly by scroll position, while a HUD depth meter counts from 0 to 3,800 metres and ocean-zone labels tick over as the reader passes each threshold.

The result is a narrative interface. Copy panels are pinned to the depth at which they make sense — the fact that 99% of sunlight is gone appears at 200 m, the pressure statistic at 1,000 m, and the booking call-to-action only at the sea floor. The visual grade, the page background colour, the HUD, and the text reveals are all derived from one shared scroll value, so nothing can drift out of sync.

Design intent

- **Tone:** sparse, expeditionary, awe. Copy is short and declarative; nothing is decorative for its own sake.
- **Visual identity:** deep navy fading to pure black with a single bioluminescent cyan accent (#00E5FF), thin extended technical sans typography and HUD-style micro-detail.
- **Interaction:** one gesture — scroll. There is no navigation menu, no carousel and no modal. The only interactive control is the final call-to-action.

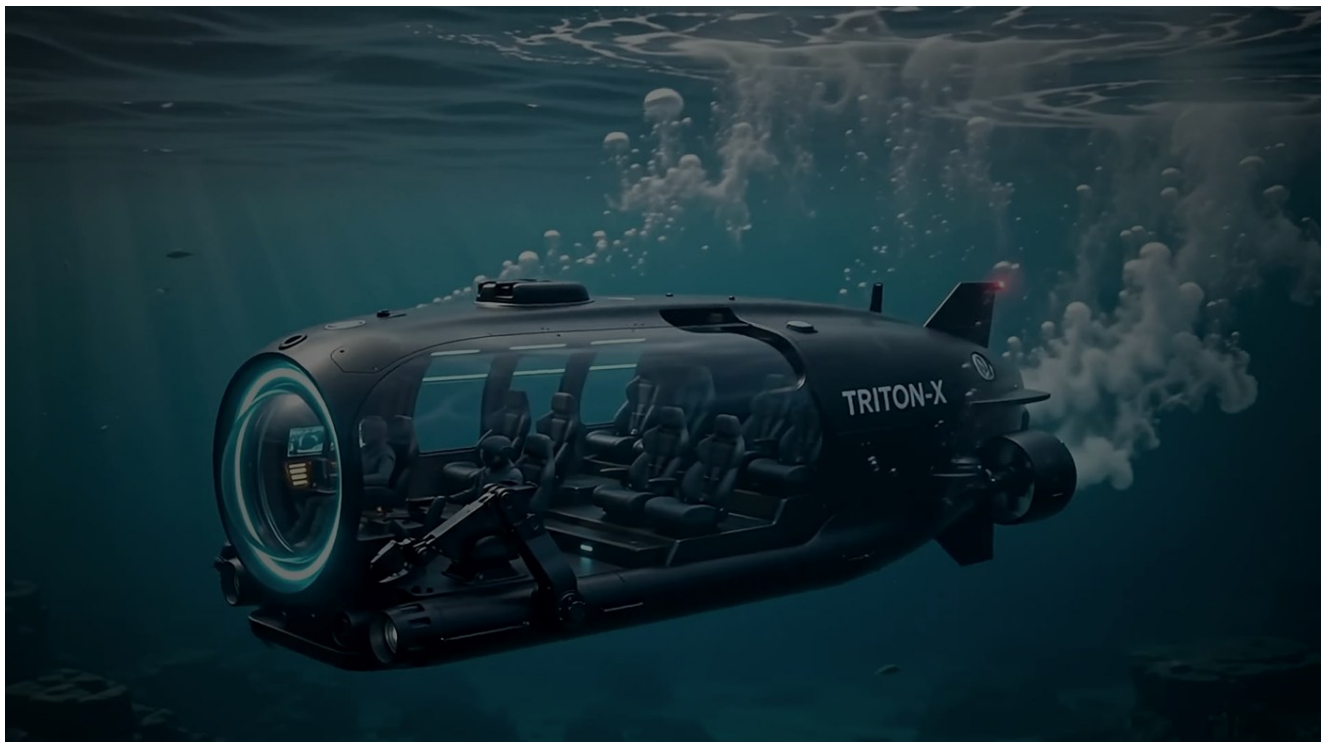


Figure 1 — The TWILIGHT zone at roughly 680 m. The canvas grade has begun pulling brightness and saturation down; the vessel's cyan viewport ring carries the frame.

02 — TECHNOLOGY

Technology stack

The project is deliberately dependency-light. There is **no package.json, no bundler, no framework and no build step** — `index.html` is served byte-for-byte identically in development and production. Every third-party library is vendored into the repository, so the site has no runtime dependency on any CDN.

Layer	Technology	Role in this project
Markup	HTML5 (semantic)	One document, 130 lines. Fixed layers plus seven scroll panels.
Styling	CSS3 (hand-written)	277 lines. Custom properties, grid, blend modes, keyframes. No preprocessor, no utility framework.
Logic	Vanilla JavaScript (ES6+)	208 lines. IIFE module, no transpilation, no polyfills.
Rendering	Canvas 2D API	Frame compositing and the live colour grade via <code>ctx.filter</code> .
Smooth scroll	Lenis 1.3.11 (vendored)	Inertial scroll normalisation across trackpad, wheel and touch.
Alternative mode	Three.js r170 (vendored)	Optional real-time WebGL descent scene; also the no-WebGL fallback path.
Typography	Syncopate, Inter, IBM Plex Mono	Google Fonts. Display, body and HUD/technical faces.
Media pipeline	FFmpeg 7.1 (via <code>imageio-ffmpeg</code>)	One-off offline grade and frame extraction.
Tooling	Python 3.12 (Pillow, NumPy)	Placeholder frame generator and report generation.
Dev server	Python <code>http.server</code>	Static file serving on localhost.
Version control	Git	4 commits on main.
Hosting	Vercel (static)	Global CDN with immutable asset caching.

A note on the backend

This project has no backend, and that is an architectural decision rather than an omission. There is no server-side runtime, no database, no API layer, no authentication and no build pipeline. Every asset — markup, styles, scripts and all 636 image frames — is a static file served directly from Vercel's CDN edge.

The consequences are worth stating explicitly:

- **No server cost or scaling concern.** Traffic is absorbed by the CDN; there is nothing to provision, autoscale or keep patched.
- **No server-side attack surface.** With no runtime, no database and no user input reaching a server, entire vulnerability classes (injection, SSRF, session hijacking) do not apply.
- **Deterministic deploys.** Because nothing is compiled, the artefact in Git is exactly what ships. A deploy is a file copy.
- **The trade-off:** the call-to-action is presentational only. Capturing real applications would require adding a form endpoint — a serverless function or a third-party form service — which is the natural first extension point.

03 — ARCHITECTURE

How the site is put together

Scroll progress is the single source of truth

Every animated property in the site is a pure function of one normalised value, `progress`, computed once per frame as `scrollY / (scrollHeight - innerHeight)` and clamped to 0–1. Because a single scalar feeds all five consumers, the visuals, the instrumentation and the copy cannot desynchronise:

- The **frame index** into the 318-frame sequence (eased toward its target for a fluid scrub).
- The **depth read-out**, via a piecewise curve.
- The **canvas colour grade** (brightness, saturation, contrast).
- The **page background**, interpolating deep navy to pure black.
- The **legibility scrim** opacity and the **[data-reveal]** text fade-and-rise.

The depth curve

Depth is not linear in scroll. A piecewise curve maps progress to metres so that each fifth of the page corresponds to one act of the descent, matching how the footage actually behaves — slow near the surface, accelerating through the mid-water column, then settling on the floor:

```
DEPTH_CURVE = [0.0 → 0 m] [0.2 → 10 m] [0.4 → 200 m]
               [0.6 → 1,000 m] [0.8 → 3,600 m] [1.0 → 3,800 m]
```

Zone labels switch at 200 m, 1,000 m and 3,500 m — deliberately the same boundaries, so the label change always coincides with a visible change in the footage.

Zone	Depth range	Scroll	Measured plate luma
SUNLIT	0 – 200 m	0 – 40%	106 → 85
TWILIGHT	200 – 1,000 m	40 – 60%	58 → 37
MIDNIGHT	1,000 – 3,500 m	60 – 92%	15 → 7
THE FLOOR	3,500 – 3,800 m	92 – 100%	9

Table 1 — Measured mean luminance of the composited canvas at each zone, sampled from the live page. The descent is monotonic, so the image always reads darker than the moment before it.

Two interchangeable renderers, one contract

The site ships two complete visual engines that honour an identical contract — the same constants, the same HUD elements and the same reveal behaviour. Swapping them is a one-line change to the script tag in `index.html`.

Renderer	Technique	Status
js/main-frames.js	Canvas 2D scrub of a pre-rendered frame sequence	Active. 208 lines.
js/dive3d.js	Real-time Three.js scene: modelled Triton-X, orbiting camera, god rays, whale, jellyfish, bioluminescent particles, hydrothermal vents	Alternative. 610 lines. Self-detects missing WebGL and injects the frame renderer as a fallback.

Maintenance note: `MAX_DEPTH`, `DEPTH_CURVE` and `ZONES` are duplicated verbatim in both files. Changing one without the other desynchronises the HUD from the visuals depending on which mode ships.

04 — MEDIA PIPELINE

From source footage to scrubbable frames

The descent footage is a user-supplied 3D animation of the Triton-X: 13.3 seconds, 3840×2160 (4K), HEVC, 24 fps. It is processed **once, offline**, into a graded master and two tiers of JPEG frames. Nothing in this pipeline runs at request time — the browser only ever downloads finished images.

The offline grade (single FFmpeg pass)

Stage	Operation	Purpose
1	delogo x=3405 y=1710 w=170 h=180	Paints out the generator's watermark by interpolating from surrounding pixels — preserves framing, unlike a crop.
2	fps=24	Normalises the source's 24.14 fps to a clean integer cadence.
3	hqdn3d=1.5:1:3:3	Light spatial/temporal denoise; removes render grain that would otherwise cost JPEG bitrate.
4	eq=contrast=1.06:saturation=1.03	Gentle contrast and saturation lift.
5	scale=1920:1080 (lanczos)	4K → 1080p downscale. Downsampling a 4K source yields a genuinely sharper 1080p than upscaling ever could.
6	unsharp=5:5:0.22	Restores micro-contrast lost to the resample.
7	vignette=angle=PI/5.5	Subtle corner falloff to focus attention on the vessel.

Quality tiers and the manifest

Frames are extracted from the graded master into two sets. `assets/frames/manifest.json` declares both, and the renderer selects a tier at load time using `innerWidth × min(devicePixelRatio, 2) > 1500`. A phone therefore downloads roughly half the bytes a desktop does, with no visible penalty at its pixel density.

Tier	Resolution	Frames	On disk	Serves
HD	1920 × 1080	318	32 MB	Desktop and retina displays
SD	1280 × 720	318	16 MB	Phones and small windows

The two-stage grade

Colour work is split between bake-time and run-time. The **static** look (denoise, sharpen, vignette, base contrast) is baked into the frames, where it costs nothing at runtime. The **depth-dependent** darkening is applied live via `ctx.filter`, because it must respond continuously to scroll.

The live grade is tuned against the footage rather than applied blindly. This clip holds its surface light until roughly 60% of the scroll and then falls sharply; a naive linear ramp left the TWILIGHT section still looking sunlit while the HUD read 680 m. The curve is therefore front-loaded — `smoothstep(0.15, 0.65, p)` driving brightness from 1.0 down to 0.39 — so the image reads as each zone at the moment the instrumentation says so.

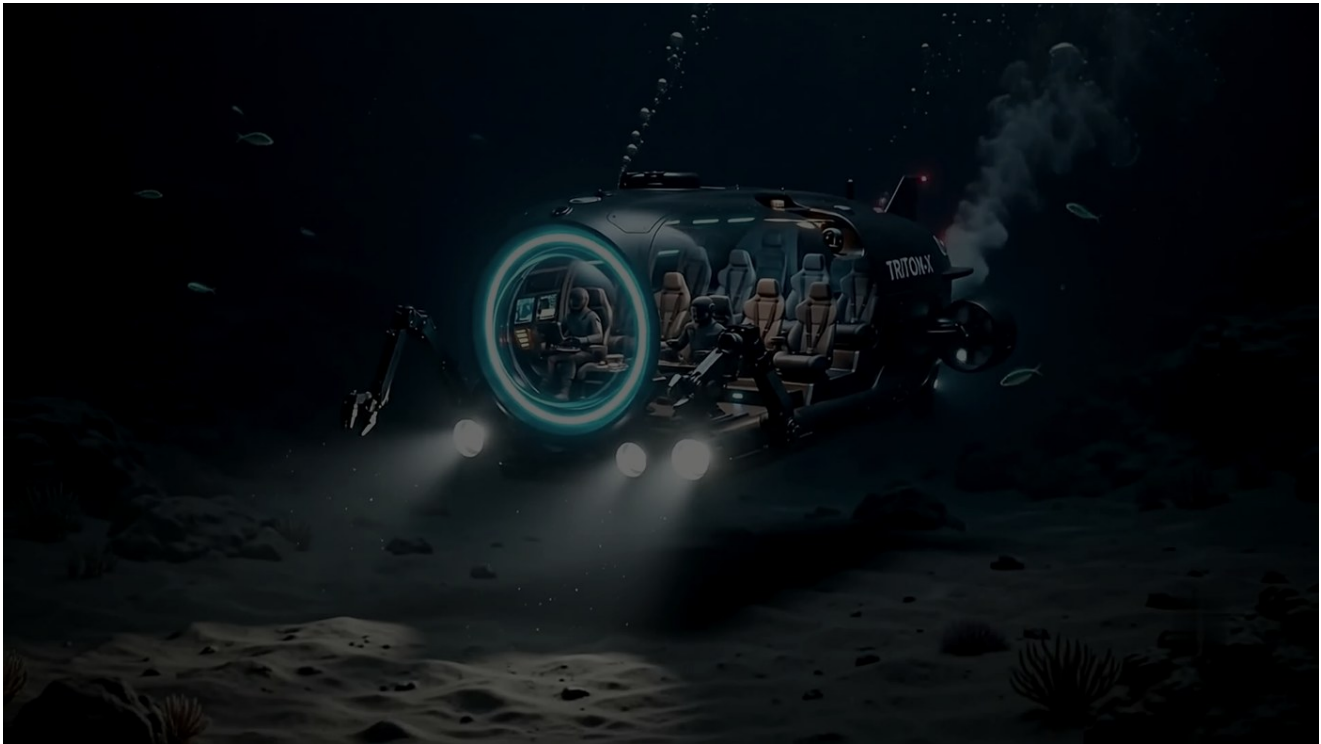


Figure 2 — THE FLOOR at 3,800 m. Mean luminance is 9/255, yet the viewport ring, cabin lights and floodlights keep the vessel legible — darkness without loss of subject.

05 — PERFORMANCE

Performance and delivery

Honest preloading

A scroll-scrubbed sequence cannot tolerate a cache miss: if the reader scrolls to a frame that has not downloaded, the animation stutters visibly. The site therefore preloads the *entire* selected tier behind a loading screen, and the progress bar reports **real** completion — it is driven by resolved image load events, not a timed animation.

Caching strategy

vercel.json sets Cache-Control: public, max-age=31536000, immutable on /assets/frames/* and /js/vendor/*. These files are content-stable, so a returning visitor revalidates nothing and the descent starts instantly. HTML, CSS and the application scripts are left on default revalidation so content changes propagate immediately; the script tags carry version query strings to defeat stale caches after a deploy.

Runtime safeguards

- **Frame-index easing.** The drawn frame eases toward its scroll target rather than snapping, smoothing both fast flicks and trackpad micro-jitter.
- **Redundant redraw suppression.** A frame is re-composited only when the frame index changes or the grade has moved by a meaningful amount.
- **Throttle resilience.** Browsers throttle requestAnimationFrame in hidden or low-power tabs, which would freeze the scrub. A scroll listener plus a 400 ms interval keep state correct while hidden.
- **Device-pixel-ratio clamp.** The canvas backing store is capped at 2× DPR, avoiding pointless 3× rasterisation on high-density phones.

Payload

Asset	Size	Notes
Frame sequence (one tier)	16 – 32 MB	Preloaded once, then cached for a year.
three.module.js (vendored)	1.3 MB	Only requested in the alternative WebGL mode.

lenis.min.js (vendored)	17 KB	Smooth-scroll runtime.
HTML + CSS + JS	~30 KB	Uncompressed application code.
Source video	44 MB	Excluded from deploy via .vercelignore.

Known trade-off: a frame sequence is inherently heavier than a video file. It is used because it permits frame-exact, instantly reversible scrubbing that no HTML5 video element reliably provides across browsers. The tiering and year-long caching are the mitigations.

Quality assurance method

Verification was performed by driving the live page programmatically rather than by eye: scrolling to each checkpoint, forcing a synchronous repaint through a debug hook (`window.__abyssalTick()`), then sampling the canvas pixel buffer to measure actual composited luminance and text contrast. Two debug hooks are left in the shipped code for this purpose — the tick function above, and `window.__abyssalSet`, which reports the chosen quality tier.

Design system and legibility

Typography and colour

Role	Face	Treatment
Display	Syncopate 700 / 400	Wide letter-spacing (0.12–0.5em). Headlines, brand mark, spec values.
Body	Inter 200 / 300	Generous line-height, letter-spaced. Lede and long-form copy.
Technical	IBM Plex Mono 300 / 400	HUD read-outs, kickers, coordinates, footnotes.

The palette is intentionally narrow: an ink white (#E8F4F8) on a background that grades from deep navy (#060D1F) to pure black, with a single bioluminescent cyan (#00E5FF) reserved for instrumentation and the call-to-action. Restricting the accent to one hue is what makes the HUD read as machinery rather than decoration.

Solving legibility over moving footage

Text over video is the hardest legibility problem in this kind of site, because the background is different in every frame. Measurement during QA found the cyan kicker, the lede, the scroll cue and the HUD rail labels all failing against the bright dusk opening. Three coordinated fixes were applied:

- **A depth-aware scrim.** A fixed gradient layer sits between the canvas and the content, heaviest at top and bottom where the chrome and HUD live. Its opacity is driven by scroll — full at the surface, easing to 30% at the floor, because the footage darkens itself and a constant scrim would crush the deep frames.
- **Text shadows** on every small type element, giving a dark halo that survives a bright frame passing behind it.
- **Raised opacity floors** on secondary copy and rail labels, and a shallower pulse range on the animated scroll cue.

Result: worst-case measured contrast on the entire page — near-white text over the brightest plate at the top of the scroll — is **8.5:1**, against the WCAG AA threshold of 4.5:1. Contrast only improves with depth as the plate darkens.

Layout and responsiveness

- **Load-bearing panel heights.** Section min-height values (140–300vh, hand-tuned per zone) exist so each section's centre lands inside its zone's slice of the scroll. Changing copy length without retuning heights silently drifts text out of sync with the depth narrative.
- **Cover-fit compositing.** Frames are drawn with a computed cover transform, so any viewport aspect ratio fills without distortion or letterboxing.
- **Mobile adaptation.** Below 760 px the HUD collapses to its rail, the mission strip is hidden, and the spec grid reflows from three columns to two.
- **Reduced motion.** A prefers-reduced-motion query disables the film-grain and pulse animations.

07 — STRUCTURE & DEPLOYMENT

Repository structure

```
sea_website/
├── index.html                130 lines – document, fixed layers, 7 panels
├── css/style.css             277 lines – full design system
├── js/
│   ├── main-frames.js       208 lines – active renderer
│   ├── dive3d.js            610 lines – Three.js alternative
│   ├── lenis.min.js         vendored smooth scroll
│   └── vendor/three.module.js
├── assets/
│   ├── frames/hd/           318 × 1920p
│   ├── frames/sd/           318 × 1280p
│   ├── frames/manifest.json
│   └── video/               source + graded master (not deployed)
├── tools/                   build_frames.sh, gen_placeholder_frames.py, snapserver.js
├── vercel.json              immutable cache headers
└── .vercelignore            excludes source video, tools
```

Deployment

The site deploys to **Vercel** as a pure static project: framework preset *Other*, no build command, no output directory. `.vercelignore` excludes the 44 MB of source video, the tooling directory and internal documentation, so only `index.html`, `css/`, `js/` and `assets/frames/` ship. Pushing to main triggers an automatic redeploy.

Local development

```
python3 -m http.server 4173    # then open http://localhost:4173
```

Rebuilding frames after replacing the footage

Re-run the FFmpeg grade against the new source, extract both tiers, and update `manifest.json` with the new frame count. **The delogo coordinates must be re-verified** whenever the source clip changes — a watermark box that no longer matches will either miss the mark or smear clean footage.

08 — REPRODUCTION PROMPT

Build this project from a single prompt

The prompt below is a complete, self-contained specification. Supplied to a capable coding agent — with one 10–15 second 4K clip of a submersible descending — it reproduces this project. It is written to be copied verbatim.

Build a single-page cinematic “scroll-dive” website for ABYSSAL VENTURES, a fictional deep-sea expedition company that takes 8 civilians a year to the ocean floor aboard its submersible, the Triton-X.

Use vanilla HTML, CSS and JavaScript with vendored libraries. No framework, no bundler, no build step.

CORE MECHANIC

Scrolling down IS descending. A fixed full-viewport canvas scrubs a pre-rendered frame sequence of a submarine descent, driven by scroll position. Extract the frames offline with FFmpeg from the video I provide: remove any watermark with the delogo filter, denoise lightly, add mild contrast, downscale to 1080p with lanczos, sharpen and vignette. Export two tiers — 1920px and 1280px — plus a manifest.json declaring both; pick HD at runtime when $\text{innerWidth} \times \min(\text{devicePixelRatio}, 2) > 1500$.

SINGLE SOURCE OF TRUTH

Derive everything from one normalised scroll progress (0–1): frame index (eased toward target), depth, canvas colour grade, page background, and text reveals. Map progress to depth with a piecewise curve — 0.2→10m, 0.4→200m, 0.6→1000m, 0.8→3600m, 1.0→3800m.

HUD

A fixed depth meter on the right edge counting 0→3,800 m with a ticked rail, a marker that tracks depth, and zone labels that flash over at 200m, 1,000m and 3,500m: SUNLIT / TWILIGHT / MIDNIGHT / THE FLOOR.

SECTIONS

Pin one panel to each zone: hero (“HOW DEEP WILL YOU GO?”) → at 200m, 99% of sunlight is gone → at 1,000m, pressure is 100× the surface → bioluminescence → Triton-X spec grid (titanium personnel sphere, 96h life support, 4,000m rating) → “8 SEATS. ₹2.4 CRORE. DEPARTING MARCH 2027.” → a JOIN THE MANIFEST call-to-action. Tune each panel's min-height so its centre lands inside its zone's slice of the scroll.

DESIGN

Page background grades deep navy (#060D1F) to pure black with depth. One bioluminescent cyan accent (#00E5FF). Thin extended technical sans — Syncopate for display, Inter for body, IBM Plex Mono for HUD micro-detail. Film grain, vignette, a real preload bar. Copy tone: sparse, expeditionary, awe.

LEGIBILITY

Text sits over bright moving footage, so add a scroll-driven scrim between canvas and content (full opacity at the surface, easing to 30% at the floor) plus text shadows on small type. Verify measured contrast stays above 4.5:1 at the brightest point.

POLISH

Lenis for smooth scroll. Ease the frame index rather than snapping. Keep the scrub correct when requestAnimationFrame is throttled in hidden tabs. Cap the canvas backing store at 2× DPR. Support prefers-reduced-motion. Deploy static to Vercel with immutable year-long caching on the frames.

VERIFY

Before declaring done, run it locally and confirm the scrub is seamless, the depth meter tracks scroll, and each section lands in its correct zone.

The prompt states outcomes and constraints rather than dictating code — which is what lets a capable agent reproduce the project while still making sound local decisions. The closing instruction matters as much as the rest: requiring the agent to run and verify before reporting completion is what separates a working site from one that merely loads.